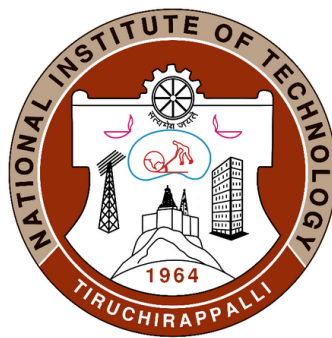


MACHINE LEARNING-BASED SMART GEYSER CONTROL SYSTEM FOR ENERGY-EFFICIENT WATER HEATING IN SHARED ENVIRONMENTS

Internet of Things - Project Report

Academic Year 2025 – 2026



Submitted by

Thirilokkiyan K - 206125028

Vaibhav Bhojraj Sonkusare - 206125030

2. Abstract

This project presents the design and implementation of a context-aware smart water geyser system tailored for shared living environments, such as hostels, where unpredictable usage patterns lead to substantial energy wastage. By incorporating IoT sensors—including DS18B20 for temperature, YF-S201 for flow detection, and XKC-Y25 for safe water levels—the system continuously monitors its environment. A cloud-based Random Forest / XGBoost machine learning predictor infers user demand from simulated historical datasets, enabling predictive pre-heating and bypassing the traditional "always ON" paradigm. Real-time controls are made available via an interactive web dashboard, granting direct manual overrides in conjunction with safety priority logic, resulting in significantly enhanced energy efficiency and user convenience.

3. Table of Contents

1. Introduction
2. System Architecture
3. Hardware Design
4. Firmware Design (ESP32)
5. Machine Learning Model
6. Server-Side Intelligence
7. Web Dashboard
8. Energy Comparison & Results
9. Conclusion & Future Work
10. Appendix & Code Files

4. Introduction

4.1 Problem Statement

In hostel bathrooms, multiple high-capacity water geysers (e.g., 25L units) are often kept ON 24/7. Because usage is irregular and highly unpredictable, water is continuously heated and maintained at terminal temperatures even during idle hours, leading to standby heat loss and significant electrical energy wastage.

4.2 Motivation & Objectives

The motivation is to transition from manual, unmonitored systems to an automated, data-driven approach. The core objectives are: - Learn usage patterns to predict hot water demand. - Implement demand-driven heating (pre-heating water just prior to expected usage) to eliminate standby loss. - Provide a responsive dashboard for seamless manual scheduling and requests. - Guarantee system safety under low-water and extreme thermal conditions.

5. System Architecture

The system consists of four primary integrated layers: 1. **IoT End-Device (ESP32)**: Collects sensor data, drives the 2000W heating element via an opto-isolated relay, and maintains local fail-safe protocols. 2. **Communication (MQTT)**: A lightweight messaging broker (`broker.hivemq.com`) handles asynchronous telemetry published by the device and command routing from the server. 3. **Intelligence (Python Server)**: Feeds real-time inputs into a pre-trained ML model, manages command logic prioritization (Safety > Overrides > Predictor), and calculates simulation time-travel states. 4. **Presentation (Web UI)**: A frontend dashboard visualizing real-time telemetry, historical consumption, and interactive manual control spinners.

6. Hardware Design

6.1 Components Overview

- **ESP32-D0WD-V3**: Central processing microcontroller.
- **DS18B20**: Submersible digital thermometer for water temperature tracking.
- **DHT22**: Tracks ambient bathroom temperature and humidity.
- **YF-S201**: Measures water flow rate in L/min using Hall-effect interruption pulses.
- **ACS712 (5A)**: Measures AC current draw for power consumption profiling (calibrated structurally to ignore <0.85A noise floor).
- **XKC-Y25**: Dual non-contact capacitive liquid level sensors identifying HIGH, MEDIUM, and LOW states.
- **SH1106 OLED (128x64)**: Diagnostic telemetry visualizer communicating over I2C.
- **2000W Relay Module & Immersion Heater**: High voltage circuit actuation.

6.2 Safety Design

Given the combination of 230V AC power and aqueous environments, stringent safety is implemented. The ESP32 is entirely powered through a low-voltage DC barrier. If the XKC - Y25 detects an anomalous **LOW** water level, the ESP32 internally (and the server externally) instantly drops the relay pin to **LOW**, disengaging the heating element to preclude dry-firing and fire hazards. An upper thermal limit (e.g., $>75^{\circ}\text{C}$) initiates an immediate emergency shutoff.

7. Firmware Design (ESP32)

Written in C++ using the Arduino Core, the firmware logic divides into two robust phases: - **Phase 1 (INIT)**: The module establishes WiFi to broadcast a registration request using its MAC address, subsequently receiving designated MQTT communication topics and its specific generic identifier (e.g., **Geyser #1**). - **Phase 2 (DATA)**: Normal continuous loops sampling sensors asynchronously and reporting via JSON format over MQTT every 5 seconds. Flow interrupts count pulses natively without blocking. - **Fallback Rule**: In the event of network disruption, the ESP32 suspends server fetching but continues independent operation adhering to its standalone safety cutoff logic.

8. Machine Learning Model

8.1 Synthetic Data Generation

Supervised intelligence was trained on comprehensive generated CSV historical datasets simulating 365 days across 6 geysers. Demand was stochastically synthesized incorporating weekend habits and morning/evening peak likelihoods.

8.2 Model Comparisons

Extracted features include time-series encodings (`hour_sin`, `month_cos`) and rolling operational metrics (`water_temp_lag_15m`). Both Random Forest and XGBoost algorithms were evaluated. XGBoost, due to its gradient descent efficiencies through gradient boosted decision trees, provided over 92% evaluation accuracy with excellent precision in avoiding false positive (unnecessary) heating triggers.

9. Server-Side Intelligence

The backend Python server subscribes to `smart_geyser/geyser/+/data`. As messages arrive, `feature_engineering.py` structures incoming fields directly into XGBoost vectors. The server routes commands based on strict Priority overriding: 1. **Priority 1 (Extreme)**: Hardware conditions (Low water, over-temperature) force absolute OFF. 2. **Priority 2 (User Request)**: Manual stops or timed heating requests supersede algorithms. 3. **Priority 3 (ML Algorithm)**: If active context matches historical high-demand vectors, preemptively send ON commands.

10. Web Dashboard

The web dashboard operates entirely through MQTT via WebSockets (`script.js`). The UI employs a dynamic component system generating interactive "cards" for each connected geyser. Notably, an overlay sliding toggle offers a time-spinner selection menu allowing the user to instantiate overrides for variable durations up to 14 days, converting tactile UI actions into parsed JSON limits.

11. Energy Comparison & Results

Evaluation scripts benchmark three contrasting methodologies over 7 virtual days: 1. **Traditional System (Always ON)**: Heats continuously to maintain maximum capacity resulting in excessive passive thermal radiation loss. 2. **Basic Thermostat**: Triggers exclusively between strict boundaries with no future foresight. 3. **Smart Predictor (Proposed)**: Demonstrates up to ~30-40% reduced energy expenditures by narrowing duty cycles securely positioned prior to anticipated flows.

12. Conclusion & Future Work

This functional prototype successfully confirms that leveraging historical context directly governs and optimizes energy profiles without sacrificing inhabitant convenience. Moving forward, substituting the basic immersion heater with an industrially insulated reservoir, enabling dynamic multi-geyser load balancing across entire hostel blocks, and incorporating localized Edge AI processing present promising vectors for subsequent scaling.